

ADRs — POS Tech Challenge Fase 3

Architecture Decision Records — decisões arquiteturais permanentes

ADR 0001 — Separação do sistema em quatro repositórios

- **Status:** Aceito
- **Data:** 2026-09-09

Contexto

O desafio exige organizar o projeto em quatro repositórios separados, cada um com CI/CD próprio e deploy automático para a nuvem: Function Serverless, infraestrutura Kubernetes, infraestrutura do banco gerenciado, e aplicação principal.

Decisão

O sistema POS é dividido em quatro repositórios independentes, cada um com seu próprio ciclo de vida de deploy:

1. ``tech-challenge-database`` — Terraform: VPC + RDS MySQL. Sem código de aplicação.
2. ``tech-challenge-lambda-functions`` — Terraform + Node.js 24: Function Serverless de autenticação (API Gateway + Lambda).
3. ``tech-challenge-kubernetes`` — Terraform (EKS) + manifests Kubernetes puros. Alvo de deploy de produção real, sem código de aplicação.
4. ``tech-challenge-application`` — Laravel 13 DDD. Contém também um `infra/` Terraform próprio, mas apontado para um ambiente Minikube local (desenvolvimento/homologação), não para a produção real em EKS.

A comunicação entre repositórios acontece exclusivamente por:

- **Terraform ``data source``** (nunca `resource`) quando um repositório precisa referenciar um recurso de outro — ex.: `tech-challenge-lambda-functions` lê a VPC/RDS de `tech-challenge-database` sem nunca gerenciá-la.
- **``repository_dispatch``** (GitHub Actions) quando um pipeline precisa disparar o pipeline de outro repositório — ver ADR 0002 (0002-repository-dispatch-para-cicd-cross-repo.md).

Consequências

- Cada repositório pode ser versionado, revisado e implantado independentemente — uma mudança de UI na aplicação não exige tocar em Terraform de infraestrutura, e vice-versa.
- O uso exclusivo de `data source` (nunca `resource`) entre repositórios de infraestrutura evita que um `terraform apply` em um repositório destrua ou reverta recursos geridos por outro — trade-off aceito: qualquer mudança estrutural (ex.: novo Security Group rule) precisa ser feita no repositório dono do recurso, nunca "de fora".
- Quatro pipelines de CI/CD independentes precisam ser mantidos e documentados (um `README.md` por repositório), aumentando a carga operacional em troca de deploys isolados e menor blast radius por mudança.

ADR 0002 — repository_dispatch como padrão de comunicação entre pipelines de CI/CD

- **Status:** Aceito
- **Data:** 2026-09-09

Contexto

Com o sistema dividido em quatro repositórios (ADR 0001 (0001-quatro-repositorios-separados.md)), o deploy de um repositório às vezes precisa disparar o deploy de outro — por exemplo, depois que a aplicação principal termina de subir no Minikube, o CI/CD da Function Serverless de autenticação deve rodar automaticamente, já que ambos compartilham segredos (jwt_secret) que precisam estar em sincronia. O GitHub Actions nativo (workflow_run) só observa workflows do **mesmo** repositório — não cruza repositórios.

Decisão

Usar o evento repository_dispatch do GitHub Actions como mecanismo padrão de disparo entre pipelines de repositórios diferentes, seguindo o padrão que já existia entre tech-challenge-application e tech-challenge-kubernetes (build-ghcr.yml → K8S_REPO_TOKEN):

- tech-challenge-application/.github/workflows/deploy-minikube.yml, no último step do job deploy, usa peter-evans/repository-dispatch@v3 para disparar o evento deploy-lambda-auth em edunsantos/tech-challenge-lambda-functions, autenticado com um PAT dedicado (LAMBDA_REPO_TOKEN).
- tech-challenge-lambda-functions/.github/workflows/deploy.yml escuta repository_dispatch: types: [deploy-lambda-auth] (além de workflow_dispatch para disparo manual).
- Cada integração cross-repo usa um **PAT dedicado por par de repositórios** (K8S_REPO_TOKEN, LAMBDA_REPO_TOKEN), nunca um token compartilhado entre todas as integrações — limita o raio de exposição caso um token vaz.

Consequências

- Novas integrações cross-repo seguem o mesmo padrão: novo secret <ALVO>_REPO_TOKEN no repositório de origem, novo repository_dispatch type documentado no workflow de destino.
- O disparo é "fire-and-forget" do lado do repositório de origem — se o pipeline de destino falhar, o repositório de origem não é notificado automaticamente; falhas precisam ser observadas via GitHub Actions do repositório de destino ou via a futura integração de observabilidade (Datadog/New Relic).
- Cada PAT precisa ser rotacionado manualmente e tem escopo repo completo sobre o repositório de destino — mitigado por serem PATs dedicados por integração, não um token de organização.

- O payload de disparo (sha, ref) permite que o workflow de destino, se necessário no futuro, correlacione o deploy disparado com o commit de origem — hoje não é usado além de log implícito no `client-payload`.

ADR 0003 — Backend remoto de Terraform (S3 + DynamoDB) por repositório

- **Status:** Aceito
- **Data:** 2026-09-09

Contexto

tech-challenge-database e tech-challenge-lambda-functions mantinham o state do Terraform local, aplicado manualmente por quem estivesse com o arquivo .tfstate na própria máquina. Isso já era arriscado com o time trabalhando localmente (state divergente entre pessoas) e se tornou inviável ao mover os dois repositórios para CI/CD em runners efêmeros do GitHub Actions, que não persistem arquivos entre execuções.

Decisão

Provisionar um backend remoto S3 + DynamoDB para eliminar o state local, fora do Terraform gerenciado pelos próprios repositórios (para evitar o problema de "ovo e galinha" de um backend remoto gerenciar seu próprio bootstrap). Na decisão original, os dois repositórios reaproveitariam o mesmo bucket/tabela — na implementação real, apenas tech-challenge-database seguiu esse plano; tech-challenge-lambda-functions provisionou seu próprio backend, sem compartilhar recursos com o outro repositório.

- Um bucket S3 versionado e criptografado (techchallenge-tfstate) usado por tech-challenge-database para armazenar seus arquivos de state.
- Uma tabela DynamoDB (techchallenge-tf-locks), chave primária LockID (string), billing PAY_PER_REQUEST, usada por tech-challenge-database exclusivamente como mutex distribuído de lock — nenhum outro atributo ou uso de dados.
- tech-challenge-lambda-functions usa um backend próprio, não compartilhado: bucket S3 tech-challenge-lambda-functions-477478162709-tfstate e tabela DynamoDB tech-challenge-terraform-locks, provisionados separadamente para este repositório (verificado em 2026-09-14).

Configuração real do backend remoto usada por tech-challenge-database (a única que segue o plano original desta ADR):

Repositório	Backend remoto (bucket • key • tabela DynamoDB)
tech-challenge-lambda-functions	tech-challenge-lambda-functions-477478162709-tfstate • tech-challenge-lambda-functions/terraform.tfstate • tech-challenge-terraform-locks
tech-challenge-database	techchallenge-tfstate • database/terraform.tfstate • techchallenge-tf-locks

```

terraform {
  backend "s3" {
    bucket      = "techchallenge-tfstate"
    key         = "database/terraform.tfstate"
    region      = "us-east-1"
    dynamodb_table = "techchallenge-tf-locks"
    encrypt     = true
  }
}

```

Configuração real do backend remoto usada por tech-challenge-lambda-functions — backend próprio, criado à parte, sem reaproveitar o bucket/tabela acima:

```

terraform {
  backend "s3" {
    bucket      = "tech-challenge-lambda-functions-477478162709-tfstate"
    key         = "tech-challenge-lambda-functions/terraform.tfstate"
    region      = "us-east-1"
    dynamodb_table = "tech-challenge-terraform-locks"
    encrypt     = true
  }
}

```

O bootstrap do bucket/tabela usados por tech-challenge-database foi feito manualmente uma única vez via AWS CLI por um humano com credenciais reais — nunca por um pipeline de CI, já que o próprio backend que os pipelines usam depende desses recursos existirem primeiro. tech-challenge-lambda-functions seguiu um caminho equivalente, mas com bucket/tabela próprios — esse bootstrap não está documentado nesta ADR.

Consequências

- terraform apply em CI (GitHub-hosted runner, sem disco persistente) passa a ser seguro — o lock via DynamoDB previne duas execuções concorrentes corromperem o mesmo state.
- tech-challenge-kubernetes **não** foi migrado para este backend nesta decisão — continua com terraform.tfstate/terraform.tfstate.backup versionados diretamente no git, um risco de exposição de segredos já identificado e registrado como pendência conhecida, fora do escopo desta ADR.
- Na prática, tech-challenge-lambda-functions não reaproveitou o bucket/tabela de tech-challenge-database — cada repositório com Terraform mantém hoje seu próprio backend remoto isolado, e não um bucket/tabela único compartilhado por todos, como esta ADR previa originalmente (verificado em 2026-09-14).
- Migração de state local pré-existente (quando havia) exige terraform init -migrate-state como passo único e manual antes do primeiro apply via CI.

ADR 0004 — Horizontal Pod Autoscaler (HPA) para escalabilidade da aplicação

- **Status:** Aceito
- **Data:** 2026-09-09

Contexto

O desafio exige um cluster Kubernetes "com escalabilidade". O cluster EKS (tech-challenge-kubernetes/eks.tf) provisiona um node group com `desired_size = 2` e `max_size = 2` nós — a escalabilidade elástica, portanto, precisa acontecer no nível de **pods** dentro da capacidade de nós já provisionada, não no nível de infraestrutura (Cluster Autoscaler), que exigiria permissões de IAM adicionais e não foi contratado neste desafio.

Decisão

Configurar um HorizontalPodAutoscaler (k8s/02-app/app-hpa.yaml) sobre o Deployment da aplicação principal (postech-app):

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: postech-app-hpa
  namespace: postech
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: postech-app
  minReplicas: 2
  maxReplicas: 4
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 60
      policies:
        - type: Pods
          value: 2
          periodSeconds: 60
    scaleDown:
      stabilizationWindowSeconds: 300
      policies:
        - type: Pods
          value: 1
          periodSeconds: 120
```

- **`minReplicas: 2`**: garante alta disponibilidade básica mesmo em carga baixa — nunca roda com um único pod.
- **`maxReplicas: 4`**: teto dimensionado para o node group atual (2 nós `t3.small`), evitando pods presos em Pending por falta de capacidade.
- **Métrica CPU 70%**: gatilho simples e suficiente para o perfil de carga de uma API REST sem processamento pesado assíncrono.
- **`behavior` assimétrico**: escala para cima rápido (60s de janela, até 2 pods por vez) para absorver picos; escala para baixo devagar (300s de janela, 1 pod por vez) para evitar oscilação (**flapping**) em carga instável.

Consequências

- A escalabilidade fica limitada pela capacidade fixa do node group (2 nós); um Cluster Autoscaler seria necessário para elasticidade além disso — registrado como possível evolução futura, fora do escopo deste desafio.
- Depende do `metrics-server` estar instalado e funcional no cluster para que a métrica de CPU exista; o `workflow deploy-eks.yml` tem uma flag `deploy_metrics_server` que precisa estar habilitada em produção real (no ambiente Minikube local ela é desligada por padrão, já que o `metrics-server` já vem junto do addon do Minikube).
- Nenhuma métrica de negócio (ex.: fila de ordens de serviço) é usada como gatilho de escala nesta fase — apenas CPU, a métrica mínima exigida para "escalabilidade" no desafio.